The Mathematics of SVM: Hard Margin

Contents

1. Overview . . . 2
2. The decision rule . . . 3
   - 2.1 A vector perpendicular to the hyperplanes . . . 3
   - 2.2 Projecting a new point onto w . . . 3
   - 2.3 The same rule in 2D . . . 5
3. The equations of π^+ and π^- . . . 5
   - 3.1 The assumption . . . 6
   - 3.2 Scaling w and b changes the margin . . . 6
4. The constraints . . . 7
   - 4.1 Green points above π^+, red points below π^- . . . 8
   - 4.2 One constraint for both classes . . . 8
5. The margin in terms of w . . . 9
   - 5.1 The derivation . . . 9
   - 5.2 The formula with numbers . . . 11
6. The optimisation problem . . . 11
7. The limit: hard-margin SVM . . . 13
8. Summary . . . 15
9. Sources . . . 15
10. Key terms . . . 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Support vector machines.
- **Compare with:** Logistic regression.

1. Overview

Key point: The SVM line $w^T x + b = 0$ has two parallel edges $w^T x + b = \pm 1$ around an empty band of width $2/\|w\|$. The band is wider the shorter w is, so SVM picks the shortest w that keeps every point on its correct side.

The margin described SVM in pictures: among all lines that separate two classes, choose the one whose margin d is the largest. The margin is the distance between the positive hyperplane π^+ and the negative hyperplane π^-: two lines (in 2D; a hyperplane is the flat boundary of any dimension) parallel to the boundary π, each touching the closest points of its class, the **support vectors** (G-1923). This Note turns that picture into a formula that a computer can optimise.

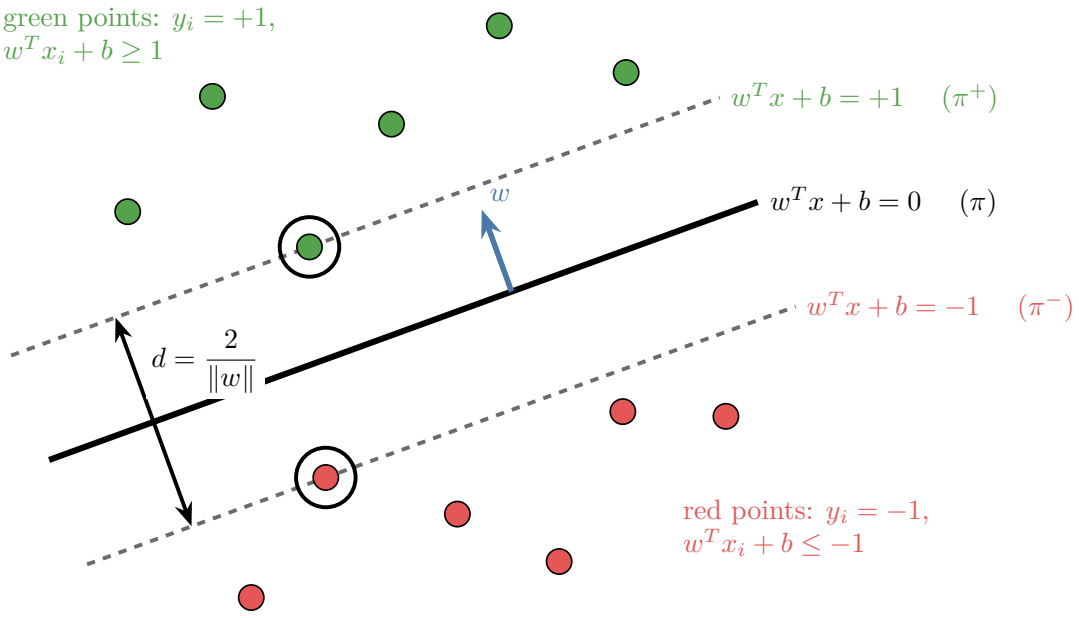


Figure 1: The three hyperplanes of an SVM, their equations, and the margin

Before the steps, here are the symbols of the Key point, each on one small instance. Take the line $2x + 3y + 3 = 0$ (as in the perceptron trick). Its numbers form the weight vector $w = (2, 3)$ (a vector is an ordered list of numbers; this one has two, one per feature) and the intercept $b = 3$. A point such as $x = (3, 2)$ is also a vector: x-coordinate 3, y-coordinate 2.

$w^T x$ (read: w transpose x, the same as $w \cdot x$) multiplies the vectors entry by entry and adds the products. The T only says "write w as a row so it can multiply x":

$$w^T x = 2 \times 3 + 3 \times 2 = 6 + 6 = 12$$

$w^T x + b$ is the score of the point: how far it sits to one side of the line, in units of w:

$$w^T x + b = 12 + 3 = 15$$

$\|w\|$ ("norm of w") is the length of the arrow w, from Pythagoras:

$$\|w\| = \sqrt{2^2 + 3^2} = \sqrt{4 + 9} = \sqrt{13} = 3.606$$

Figure 1 shows the result. The steps to get there:

1. A **decision rule** that says on which side of the line a new point lies.
2. Equations for π^+ and π^-, and why they use $+1$ and -1.
3. **Constraints** that keep every training point on its correct side.
4. A formula for the margin d in terms of w.
5. The optimisation problem that puts it all together.

2. The decision rule

> **Key point:** Compute the point's score. If the score is zero or positive, predict $+1$ (green); if it is negative, predict -1 (red). The score is $w \cdot u + b$.

Before finding the best line, we need to know how a line classifies a new point once we have it.

2.1 A vector perpendicular to the hyperplanes

> **Key point:** π, π^+ and π^- are parallel, so one vector w is perpendicular to all three. At first we know its direction but not its length.

The three hyperplanes are parallel. So a vector w that is perpendicular to one of them is perpendicular to all three (the blue arrow in Figure 1). At this point we only fix the **direction** of w; its length is still unknown and will be decided later.

2.2 Projecting a new point onto w

> **Key point:** Project the new point u onto w. If the projection passes a threshold c, the point is positive.

Take an unknown point u. In 2D a point is also a vector from the origin, with an x and a y component. Drop u onto the direction of w: this is the projection of u onto w, and its length grows with the dot product $w \cdot u$ (Figure 2).

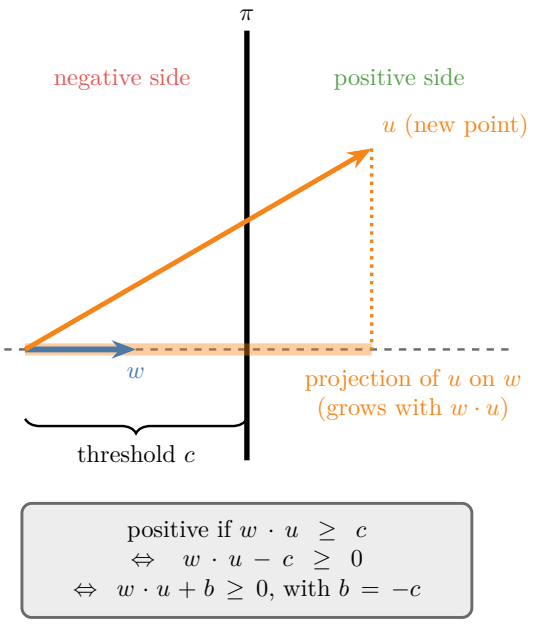


Figure 2: The decision rule: a point is positive when its projection onto w passes the threshold c

A worked instance with $w = (2, 3)$ and $u = (3, 2)$. The dot product is

$$w \cdot u = 2 \times 3 + 3 \times 2 = 12$$

and the projection of u onto the direction of w has length (the dot product divided by the length $\|w\| = 3.606$ of w, see Section 5)

$$12/3.606 = 3.33$$

The boundary π crosses the direction of w at some distance from the origin. Call c the dot-product value at which π is crossed; for the line $2x + 3y + 3 = 0$ the crossing is at $c = -3$. A point is on the positive side when its dot product goes past that threshold. For $u = (3, 2)$ the dot product is 12, which is above -3, so u is positive:

$$12 \geq -3$$

In general:

$$w \cdot u \geq c$$

$$w \cdot u - c \geq 0$$

$$w \cdot u + b \geq 0, \quad \text{with } b = -c$$

So the **decision rule** (G-558) of SVM is:

$$\hat{y} = \begin{cases} +1 & \text{if } w \cdot u + b \geq 0 \\ -1 & \text{if } w \cdot u + b < 0 \end{cases}$$

Once we know w and b, classifying any new point takes one dot product and one addition.

2.3 The same rule in 2D

> **Key point:** In 2D, w · u + b is just "put the point into the line's equation", as in the perceptron trick.

In 2D this rule is the side test of **positive and negative sides** (G-1529): put the point into the left-hand side of the line's equation and look at the sign. Written with vectors, the line $2x + 3y + 3 = 0$ has $w = (2, 3)$ and $b = 3$.

- $u = (3, 2)$:

$$w \cdot u + b = 2 \times 3 + 3 \times 2 + 3 = 15$$

  This is at least 0, so the point is **positive**.

- $u = (-2, 0)$:

$$w \cdot u + b = 2 \times (-2) + 3 \times 0 + 3 = -1$$

  This is below 0, so the point is **negative**.

The vector form says exactly the same thing, but it works unchanged in any number of dimensions.

3. The equations of π^+ and π^-

> **Key point:** We choose π^+ to be $w^T x + b = +1$ and π^- to be $w^T x + b = -1$. Equal sizes keep π in the middle; the value 1 is a free choice of scale.

3.1 The assumption

> **Key point:** π: $w^T x + b = 0$, π^+: $w^T x + b = +1$, π^-: $w^T x + b = -1$.

To compute the margin we need equations for π^+ and π^-. We **choose** them to be:

$$\pi^+ : \; w^T x + b = +1$$

$$\pi : \; w^T x + b = 0$$

$$\pi^- : \; w^T x + b = -1$$

For the line $2x + 3y + 3 = 0$, these are $2x + 3y + 3 = 1$ and $2x + 3y + 3 = -1$: two lines parallel to it, one on each side. This choice raises three questions.

1. **Why must the two right-hand sides have the same size** (for example not $+1$ and -2)? Because π must lie exactly in the middle of π^+ and π^-. The distance from π to π^+ must equal the distance from π to π^-, and equal values on both sides give that.
2. **Why 1, and not 25 or 37?** The exact number makes no difference to which line is best. Section 5 shows that a different number only multiplies the margin by a constant, so 1 is chosen for convenience.
3. **Why are these equations useful at all?** Section 3.2 answers this with an experiment.

3.2 Scaling w and b changes the margin

> **Key point:** Multiplying w and b by 10 gives the same line π, but π^+ and π^- move closer together. Dividing by 10 pushes them apart.

Multiply every coefficient of $2x + 3y + 3 = 0$ by 10. The result, $20x + 30y + 30 = 0$, is the **same line**: the slope and intercept do not change.

Now do the same to π^+ and π^-, multiplying only the left-hand side. The equations $20x + 30y + 30 = \pm 1$ are **not** the same lines as $2x + 3y + 3 = \pm 1$. Since the right-hand side stays at 1, the new π^+ and π^- lie much closer to π (Figure 3).

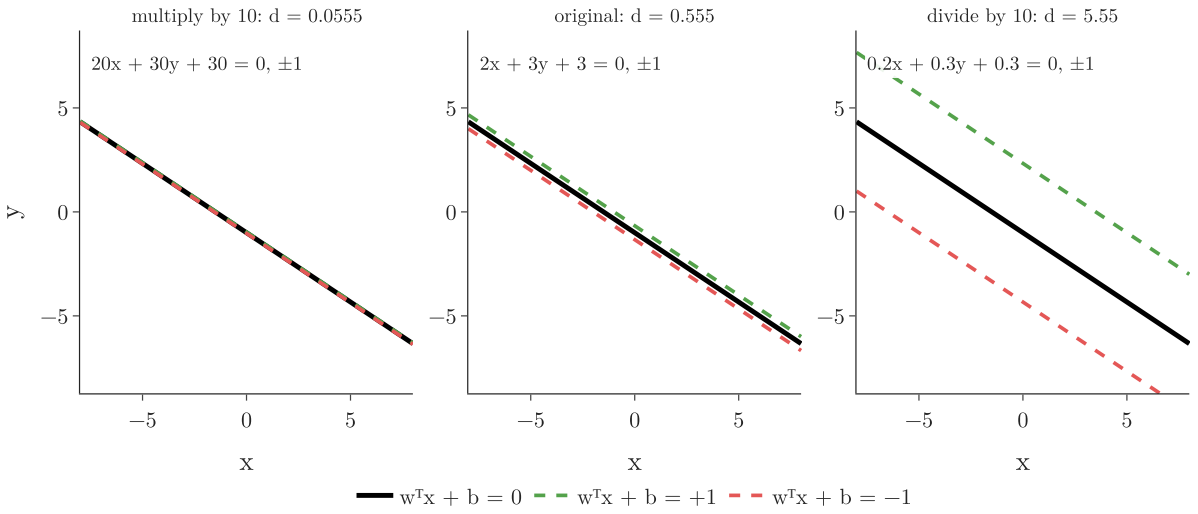


Figure 3: The same line π with w and b multiplied by 10, unchanged, and divided by 10

In plain words: scaling w and b up squeezes the margin; scaling them down widens it. As a formula (derived in Section 5), with $\|w\|$ the length of w:

$$d = \frac{2}{\|w\|}$$

With numbers, for w equal to k times $(2, 3)$:

Factor k	Equation of π	$\|w\|$	Margin d
10	$20x + 30y + 30 = 0$	36.06	0.0555
1	$2x + 3y + 3 = 0$	3.606	0.555
0.1	$0.2x + 0.3y + 0.3 = 0$	0.3606	5.55

The scaling effect is why the ± 1 equations are useful. While an optimiser adjusts w and b, it can both turn and shift the line and make the margin wider or narrower. The search for the best line becomes a search over w and b alone.

4. The constraints

> **Key point:** Every training point must lie on or beyond its own hyperplane: $y_i(w^T x_i + b) \geq 1$ for every i.

4.1 Green points above π^+, red points below π^-

> **Key point:** Green: $w^T x + b \geq 1$. Red: $w^T x + b \leq -1$. **Support vectors** (G-1923) give exactly ± 1.

Each training point x_i is one **observation** (G-1374; one record, a row of the data table). Its coordinates are its **features** (G-772; input variables, one column each), and its class is the **target** (G-1949; the output we predict).

The margin only means something if no point sits inside it. π^+ and π^- stop at the first point of each class, so no green point can lie below π^+ and no red point above π^-. In equations:

- every green point: $w^T x_i + b \geq 1$;
- every red point: $w^T x_i + b \leq -1$.

The support vectors lie exactly on π^+ or π^-, so they give exactly $+1$ or -1. All other points give more than $+1$ or less than -1.

4.2 One constraint for both classes

> **Key point:** Label green points $+1$ and red points -1, and multiply: both rules become $y_i(w^T x_i + b) \geq 1$.

Two separate rules are awkward to work with, so we merge them. Give every green point the target value $y_i = +1$ and every red point the target value $y_i = -1$, then multiply $w^T x_i + b$ by the target value.

- **Green point:** $y_i = +1$ leaves $w^T x_i + b \geq 1$ unchanged.
- **Red point:** multiplying $w^T x_i + b \leq -1$ by -1 flips the sign: $-(w^T x_i + b) \geq 1$.

Both become one **constraint** (G-456), for every training point $i = 1, \ldots, n$:

$$y_i \left(w^T x_i + b \right) \geq 1$$

with equality for the **support vectors** (G-1923). Figure 4 checks this on the data of the margin of a line, whose best line is $w = (0.049, 0.898)$, $b = -4.485$. For the red point $(2.5, 3)$, whose target is $y_i = -1$, the steps are:

$$w^T x + b = 0.049 \times 2.5 + 0.898 \times 3 - 4.485$$

$$= 0.1225 + 2.694 - 4.485 = -1.67$$

$$y_i(w^T x_i + b) = (-1) \times (-1.67) = 1.67 \geq 1$$

The constraint holds.

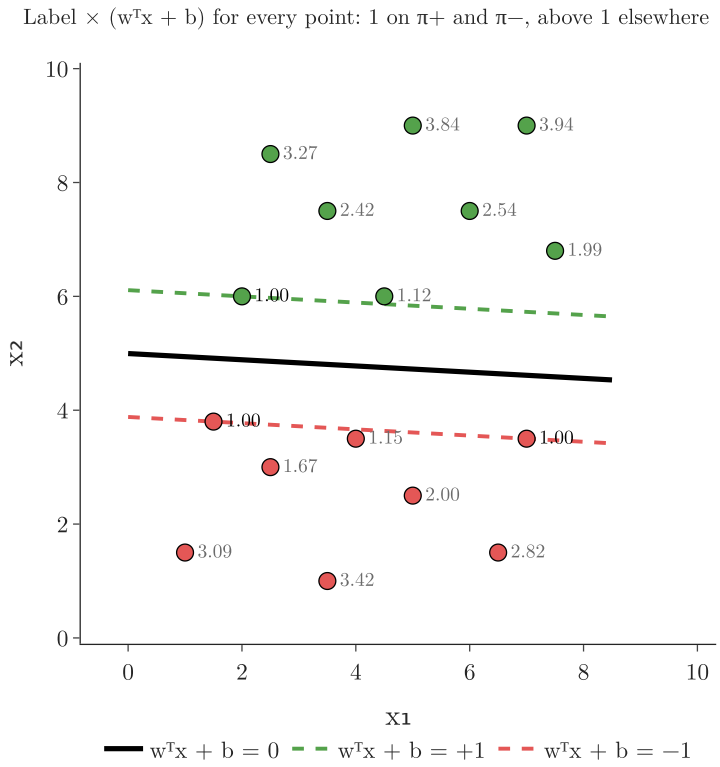


Figure 4: $y_i(w^T x_i + b)$ for every training point: exactly 1 for the three support vectors, more than 1 for the rest

The whole formula for the margin relies on these constraints. If a red point entered the region above π^-, or a green point the region below π^+, the margin would no longer be the gap between the classes.

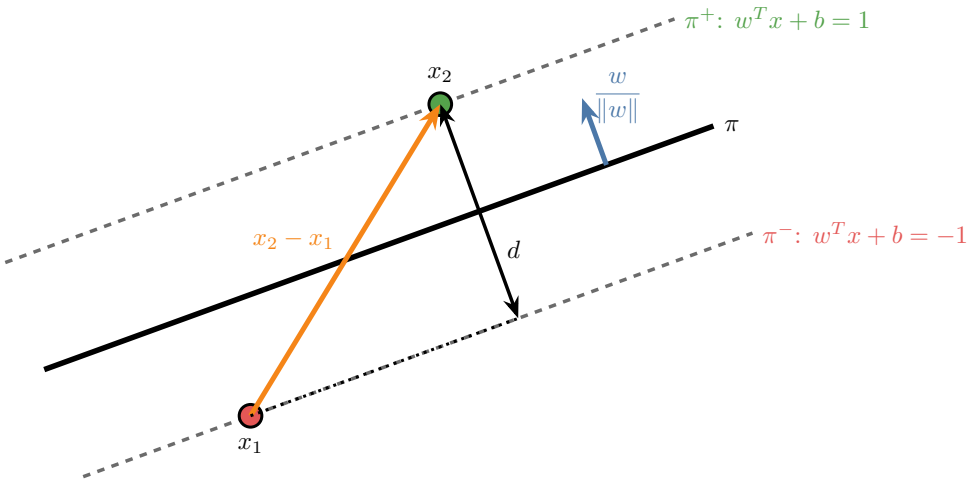
5. The margin in terms of w

> **Key point:** Project the vector between two **support vectors** (G-1923) onto the unit vector $w/\|w\|$. The result is $\mathrm{d} = 2/\|w\|$.

5.1 The derivation

> **Key point:** the margin is a projection, and the equations of π^+ and π^- turn it into $2/\|w\|$:
>
> $$d = (x_2 - x_1) \cdot w/\|w\|$$

Take one **support vector** (G-1923) on each edge: x_1 on π^- and x_2 on π^+ (Figure 5). The vector from x_1 to x_2 is $x_2 - x_1$. The vector crosses the margin, but at a slant, so its length is not the margin.



$$d = (x_2 - x_1) \cdot \frac{w}{\|w\|} = \frac{w^T x_2 - w^T x_1}{\|w\|} = \frac{(1-b) - (-1-b)}{\|w\|} = \frac{2}{\|w\|}$$

Figure 5: The margin is the projection of $x_2 - x_1$ onto the unit vector $w/\|w\|$

The **shortest** distance between the two edges is measured perpendicular to them, that is along w. So we project $x_2 - x_1$ onto the **unit vector** (G-2048) in the direction of w, which is w divided by its length (its **norm**, here the **L2 norm**, G-1028) $\|w\|$:

$$d = (x_2 - x_1) \cdot \frac{w}{\|w\|}$$

$$= \frac{w^T x_2 - w^T x_1}{\|w\|}$$

Both points are support vectors, so their equations are known:

- x_2 is on π^+: $w^T x_2 + b = 1$, so $w^T x_2 = 1 - b$.
- x_1 is on π^-: $w^T x_1 + b = -1$, so $w^T x_1 = -1 - b$.

Substituting:

$$d = \frac{(1-b) - (-1-b)}{\|w\|}$$

The top simplifies, because the two b cancel:

$$(1-b) - (-1-b) = 1 - b + 1 + b$$

$$= 2$$

So

$$d = \frac{2}{\|w\|}$$

5.2 The formula with numbers

> **Key point:** For the best line of the earlier example, $\|w\| = 0.899$ and d $= 2/0.899 =$ 2.22.

The margin is two divided by the length of w. For the line of Figure 4, $w = (0.049, 0.898)$:

$$\|w\| = \sqrt{0.049^2 + 0.898^2}$$

$$= 0.899$$

$$d = \frac{2}{0.899} = 2.22$$

The value 2.22 is the margin d measured in the margin of a line. The b cancelled out: the margin depends only on w. The margin d is the full width between π^+ and π^-; the one-sided distance from π to the nearest point, called the margin in the weakness of the perceptron, is half of it, $1/\|w\|$.

> **Extra:** The derivation also answers "why 1?". With $\pi^{\pm} : w^T x + b = \pm k$, the same steps give $d = 2k/\|w\|$. The new margin is the old one times a constant, so the same w and b are best. Equivalently, dividing w and b by k turns $\pm k$ back into ± 1 without moving any line. Fixing the **support vectors** (G-1923) at exactly ± 1 simply picks one scale for w and b.

6. The optimisation problem

> **Key point:** Find w and b that maximise $2/\|w\|$, subject to $y_i(w^T x_i + b) \geq 1$ for every point.

Putting the pieces together, SVM looks for the w and b that make the margin as large as possible while every point respects its **constraint** (G-456). Two symbols need an instance first:

- **max versus arg max.** max returns the largest value of a function; arg max returns the input that produces it. For the margin $2/\|w\|$, the best w is $(0.049, 0.898)$:

$$2/0.899 = 2.22$$

Another allowed w gives a smaller value, say:

$$2/1.2 = 1.67$$

The max is 2.22; the arg max is the vector $w = (0.049, 0.898)$ that gave it.

- w^*, b^* (read: w star) name the winning values.
- **such that ... for all** i means the condition must hold for $i = 1, 2, \ldots, n$, once for each of the n training points (here $n = 16$).

The full problem:

$$w^*, b^* = \arg\max_{w,\, b} \frac{2}{\|w\|}$$

such that

$$y_i \, (w^T x_i + b) \geq 1 \quad \text{for all } i$$

The problem is a **constrained optimisation** (G-455) problem: we maximise a function while keeping a condition true, one condition per training point. On the data of Figure 4 the answer is $w = (0.049, 0.898)$, $b = -4.485$ and $d = 2.22$.

Figure 6 shows the search in two dimensions. Every direction of w is one candidate. For each direction, the widest margin that still obeys every constraint is the gap between the lowest green point and the highest red point, measured along w. Sweeping the direction from 60 to 120 degrees traces a curve with a single peak: 2.22, at the direction of the SVM's w. The optimiser finds this peak without drawing the curve.

Maximising $2/\|w\|$ is the same as minimising $\|w\|$: the smaller the length of w, the wider the margin. From maximising to minimising uses this minimising form, and gives the standard textbook version, $\frac{1}{2}\|w\|^2$.

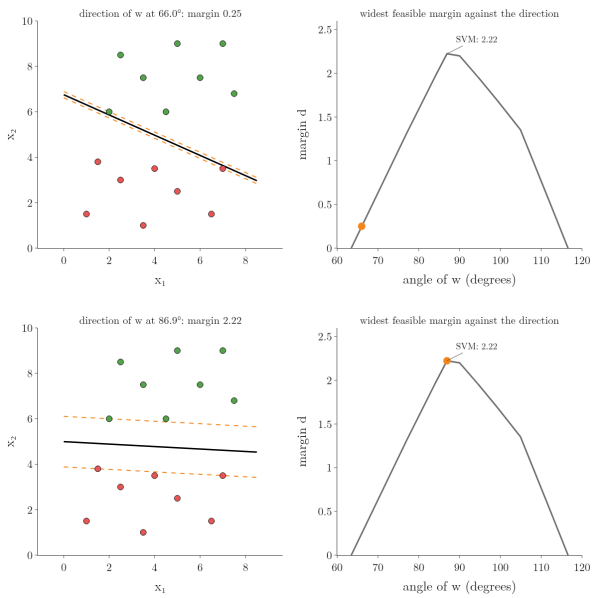


Figure 6: The optimisation as a search over directions of w, on the 16 points of the earlier example. Left: for one direction, the line in the middle of the widest gap it allows (dashed: π^+ and π^-). Right: that widest margin against the angle of w; the maximum, 2.22, is at the direction of the SVM's w.

> **Python:** scikit-learn solves this problem for us. A very large `C` makes `SVC` behave like the **hard-margin SVM** (G-879). `kernel="linear"` keeps the straight hyperplane of this Note; other kernels first lift the data into more dimensions, where a flat hyperplane can separate classes that no straight line separates in the original features. That lifting is the **kernel trick (G-1008)** (taught in full later).
>
> ```
> import numpy as np
> from sklearn.svm import SVC
>
> svm = SVC(kernel="linear", C=1e6).fit(X, y)
> w, b = svm.coef_[0], svm.intercept_[0]
> y * (X @ w + b)                  # every value >= 1
> 2 / np.linalg.norm(w)            # the margin: 2.22
> ```
>
> The Notebook of this Note checks every number in this Note and rebuilds the scaling experiment of Figure 3 with a slider.

7. The limit: hard-margin SVM

> **Key point:** The formulation needs perfectly separable data. A single point on the wrong side makes the **constraints** (G-456) impossible to meet.

Real data is often not perfectly separable (ISL §9.1.5). Usually such data is **almost** linearly separable, with a few points on the wrong side, or not linearly separable at all.

The constraints above allow no exceptions. In Figure 7 one green point lies among the red

points: for the old line it gives $y(w^T x + b) = -2.47$, far below 1. No straight line can put that point on the green side without putting red points there too, so **no** w and b satisfy every constraint. The optimisation has no answer at all.

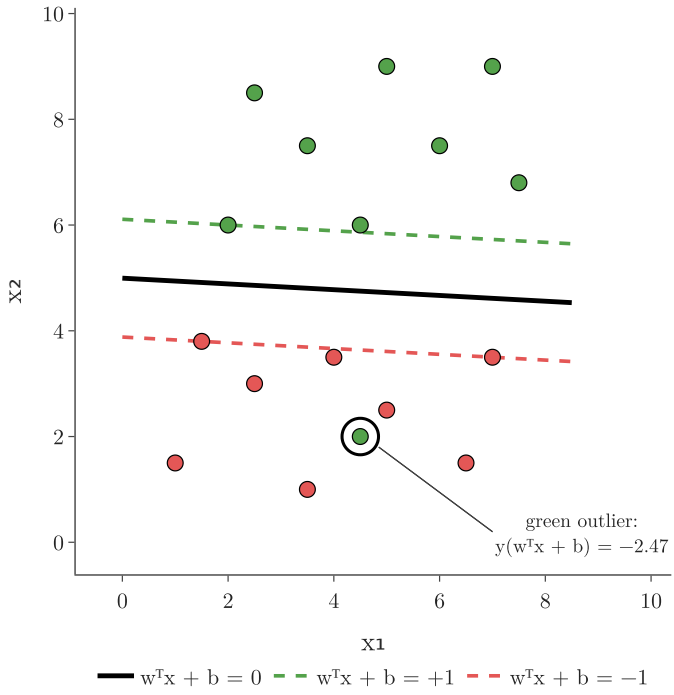


Figure 7: A single green outlier among the red points breaks the constraints of every line

The formulation of this Note is called the **hard-margin SVM** (G-879): it works only on data that a hyperplane separates perfectly. The soft margin relaxes the constraints to allow a few mistakes, which gives the **soft-margin SVM**.

8. Summary

Piece	Formula	Why it matters
Decision rule	$\hat{y} = +1$ if $w \cdot u + b \geq 0$, else -1	once w and b are known, a new point takes one dot product and one addition
Hyperplanes	$\pi : w^T x + b = 0$, $\pi^{\pm} : w^T x + b = \pm 1$	equal sizes keep π in the middle; 1 only fixes the scale of w and b
Constraint (every point)	$y_i(w^T x_i + b) \geq 1$, equal to 1 for support vectors	no point may sit inside the band, so the margin really is the gap between the classes
Margin	$d = 2/\|w\|$	b cancels, so the margin depends only on the length of w
Optimisation	maximise $2/\|w\|$ subject to the constraints	the best line is found by an optimiser instead of trial and error

- w is perpendicular to all three hyperplanes, because they are parallel; b shifts them.
- Scaling w and b leaves π in place but moves π^+ and π^-: a smaller $\|w\|$ means a wider margin, so the search for the best line becomes a search over w and b alone.
- The target values ± 1 merge the two class rules into one **constraint** (G-456), so one formula covers every point.
- **Hard-margin SVM** (G-879) fails as soon as one point is on the wrong side, because then no w and b satisfy every constraint; the soft margin fixes this.
- So the picture of the widest band becomes a problem a computer can solve: find the shortest w that keeps every point on its correct side.

9. Sources

Built from

- CampusX, "Mathematics of SVM | Support Vector Machines | Hard margin SVM", YouTube, https://www.youtube.com/watch?v=yCAlHPDgWtM

Other references

- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd edition. Springer, 2021. Section 9.1.5, p. 373.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Decision rule (SVM) (G-558)	How a trained SVM labels a new point u: by which side of the separating line it falls on, predicting $+1$ if $w \cdot u + b \geq 0$ and -1 otherwise.
Constraint (G-456)	A condition the solution must satisfy; in SVM, $y_i(w^T x_i + b) \geq 1$ for every training point.
Hard-margin SVM (G-879)	The SVM that allows no point inside the margin or on the wrong side; it needs perfectly separable data.
Support vectors (G-1923)	The training points that lie on the two margin lines, π^+ or π^-; they alone fix the SVM line.
Positive and negative side (G-1529)	The two halves of the plane on either side of a line $Ax + By + C = 0$: where $Ax + By + C$ is above 0 and where it is below 0; the perceptron predicts one class on each side.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Unit vector (G-2048)	A vector of length 1, used to describe a direction.
L2 norm (norm, magnitude, length of a vector) (G-1028)	The usual magnitude of a vector, its distance from the origin: the square root of the sum of squared components, $\|w\| = \sqrt{w_1^2 + w_2^2 + \ldots}$.
Constrained optimisation (G-455)	Maximising or minimising a function while keeping one or more constraints true.
Kernel trick (G-1008)	Making non-linear data separable by mapping it to a higher dimension, without building the new columns.